**NAME:-Shubham Suryakant Biradar**
**ROLL NO.:-A-12**
**BATCH:-A1**
**ASSINGMENT:-5**

Aim:-Data Visualization with Matplotlib: Data visualization on a dataset using matplotlib and seaborn libraries • Scatter plot • Line plot • Bar plot • Histogram • Box plot • Pair plot

THEORY:- Q.1.What is the importance of data visualization in data analysis? How does visualization **help in exploratory data analysis (EDA)?** **Answer:-**Importance of Data Visualization in Data Analysis: Helps to present data in a visual form (charts, graphs, plots) Makes complex data easy to understand Allows quick identification of patterns and trends Improves decision-making by showing clear insights Helps in communicating results effectively to others

How Visualization helps in Exploratory Data Analysis (EDA): Helps to detect missing values and outliers Shows relationships between variables (e.g., scatter plots) Identifies distribution of data (e.g., histograms) Reveals patterns, trends, and anomalies Supports better feature selection for machine learning

Q.2.What is the difference between Matplotlib and other visualization libraries? **Answer:-**The main difference between Matplotlib and other visualization libraries lies in ease of use, features, and level of abstraction.

Matplotlib is a basic and highly customizable plotting library in Python. It gives you full control over every element of a graph (like axes, labels, colors), but because of that, it often requires more code and can be harder for beginners. It is mainly used for static plots such as line charts, bar graphs, and histograms.

On the other hand, other libraries like Seaborn, Plotly, and Bokeh are built on top of or alongside Matplotlib and provide higher-level interfaces. These libraries are:
Easier to use (less code required) More visually attractive by default Better for advanced or interactive visualizations
For example: Seaborn is great for statistical graphs with beautiful themes. Plotly and Bokeh allow interactive charts (zoom, hover, click).

Q.3.What is the purpose of the pyplot module in Matplotlib? **Answer:-**The **matplotlib.pyplot** module is used to create and manage plots in a simple and user-friendly way. It provides a MATLAB-like interface that allows users to quickly generate different types of graphs such as line plots, bar charts, histograms, and scatter plots with minimal code. The pyplot module also helps in customizing plots by adding titles, labels, legends, and grids, and it allows users to display or save the graphs easily. Overall, its main purpose is to make data visualization quick, simple, and efficient without dealing with complex programming details.

Q.4.Explain the process of creating a basic line plot. **Answer:-**Creating a basic line plot using **matplotlib.pyplot** involves a few simple steps. First, you import the pyplot module. Next, you prepare your data by defining two sets of values—one for the x-axis and one for the y-axis. Then, you use the plot() function to draw the line connecting the data points. After that, you can add labels for the x-axis and y-axis and give a title to the graph to make it more understandable. Optionally, you can add a grid or legend for better clarity. Finally, you display the plot using the show() function. This process allows you to quickly visualize data in the form of a simple line graph.

Q.5.Why is customization important in data visualization? What customization options are available **in Matplotlib?** **Answer:-**Customization is important in data visualization because it makes graphs clear, meaningful, and easy to understand. Without customization, charts may look confusing or fail to highlight important information. By adjusting elements like colors, labels, and styles, we can improve readability, emphasize key insights, and make the visualization more attractive and professional. Customization also

helps in presenting data according to the audience's needs, making it easier to communicate findings effectively.

In Matplotlib, there are many customization options available, such as:

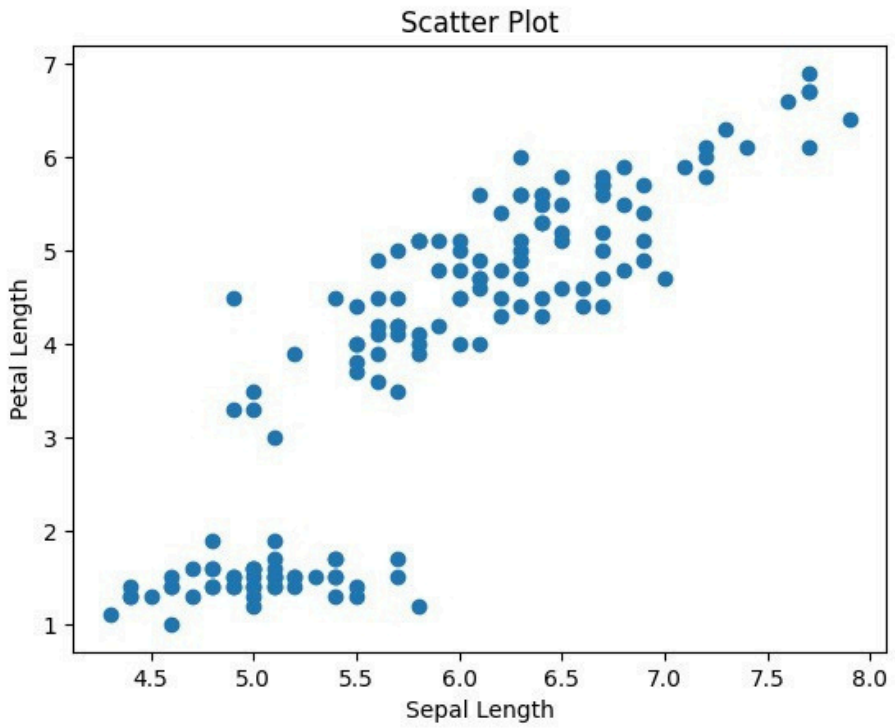
- **Titles and labels**: Add titles and name the x-axis and y-axis
- **Colors and styles**: Change line colors, markers, and line styles
- **Legends**: Describe multiple data lines in a graph
- **Grid**: Add grid lines for better readability
- **Axis limits**: Set the range of x and y axes
- **Markers**: Highlight data points with symbols
- **Figure size**: Adjust the size of the graph
- **Annotations**: Add notes or highlight specific points

Overall, customization in Matplotlib helps turn simple graphs into clear, informative, and visually appealing visualizations.

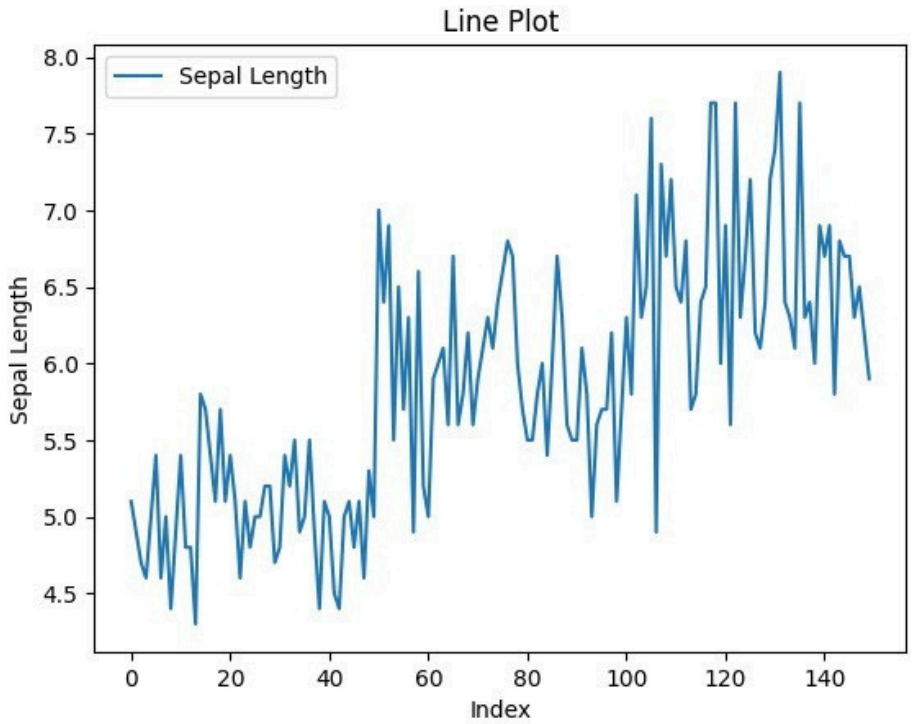
```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
df = sns.load_dataset('iris')
print(df.head())
```

```
   sepal_length  sepal_width petal_length  petal_width species
0           5.1          3.5           1.4          0.2 setosa
1           4.9          3.0           1.4          0.2 setosa
2           4.7          3.2           1.3          0.2 setosa
3           4.6          3.1           1.5          0.2 setosa
4           5.0          3.6           1.4          0.2 setosa
```

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd

# 1. Scatter Plot

df = sns.load_dataset('iris') # Ensure df is defined

plt.figure()
plt.scatter(df['sepal_length'],  df['petal_length'])
plt.title("Scatter        Plot")
plt.xlabel("Sepal       Length")
plt.ylabel("Petal       Length")
plt.show()
```

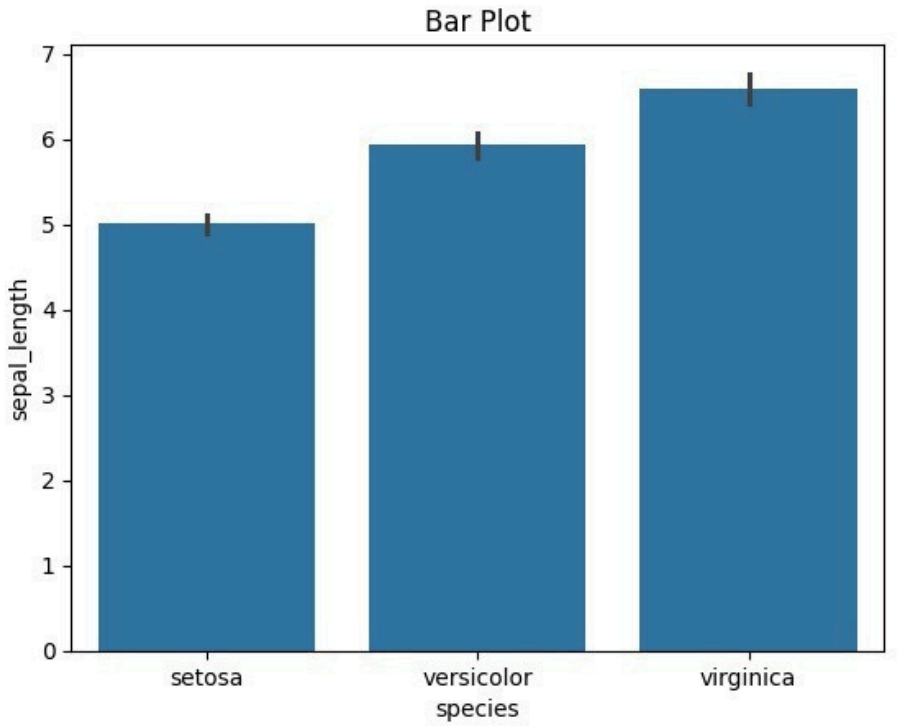



```
# 2. Line Plot
# ------------------------------
plt.figure()
plt.plot(df['sepal_length'], label='Sepal Length')
plt.title("Line Plot")
plt.xlabel("Index")
plt.ylabel("Sepal Length")
plt.legend()
plt.show()
```

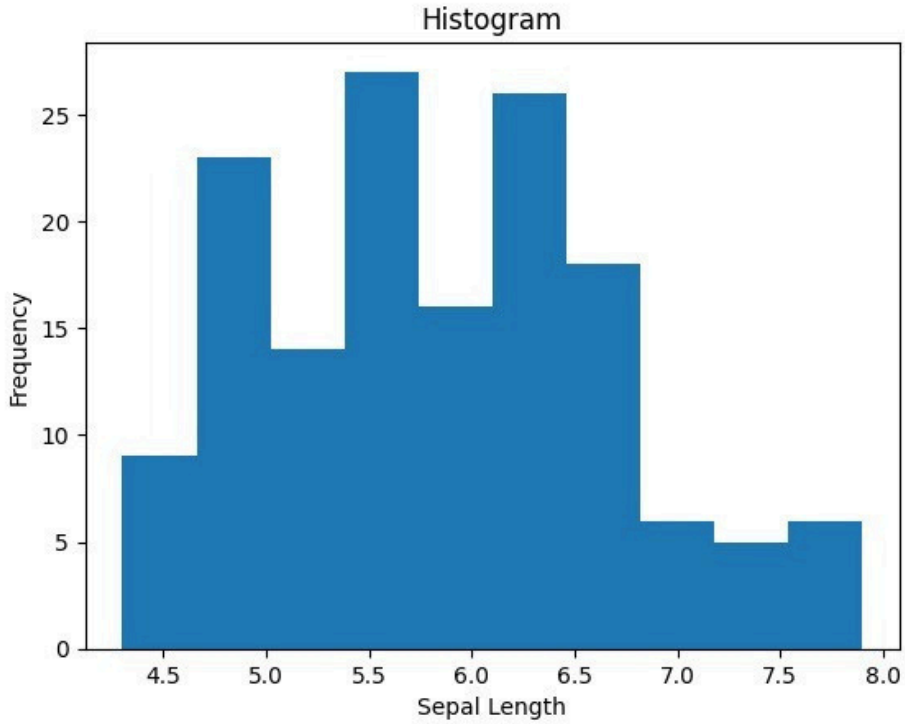



```
# 3. Bar Plot
# ------------------------------
plt.figure()
sns.barplot(x='species', y='sepal_length', data=df)
```

```
plt.title("Bar Plot")
plt.show()
```

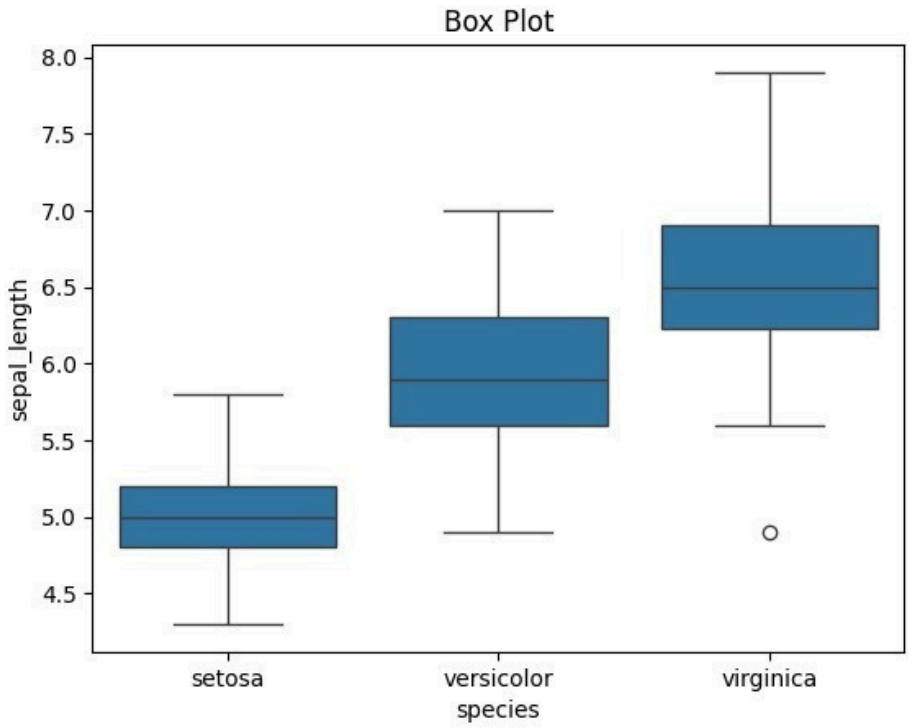



```
#4. Histogram
# ------------------------------
plt.figure()
plt.hist(df['sepal_length'], bins=10)
plt.title("Histogram")
plt.xlabel("Sepal Length")
plt.ylabel("Frequency")
plt.show()
```




```
# 5. Box Plot
# ------------------------------
plt.figure()
sns.boxplot(x='species', y='sepal_length', data=df)
```

```
plt.title("Box Plot")
plt.show()
```




**CONCLUSION:-** In this aim, data visualization was successfully performed using Matplotlib and Seaborn. Various plots such as scatter plot, line plot, bar plot, histogram, box plot, and pair plot were created to represent the dataset in different ways. These visualizations helped in understanding relationships between variables, identifying patterns and trends, analyzing data distribution, and detecting outliers. Overall, the use of Matplotlib and Seaborn made the data more meaningful and easier to interpret, proving that visualization is an essential step in data analysis and decision-making.